

6주차 과제D

사람 백소정

진행 상황

모듈	상태
정적 분석 (<code>src/static_analyzer/</code>)	✅ 완료
동적 분석 (Frida)	🚧 진행 중 (<code>feature/dynamic-agent</code> , A/B/C/D 4일차까지 완료, 5일차 전원 통합 남음)
네트워크 분석	✅ 완료 (<code>feature/network-analyzer</code> , 실제 에뮬레이터+정상 앱으로 A → B/C → D 전체 파이프라인 end-to-end 검증까지 완료)
웹 대시보드	예정

네트워크 분석 모듈 (완료)

앱이 어떤 도메인/서버와 통신하는지 tcpdump로 캡처하고, DNS 쿼리와 TLS SNI를 파싱해서 화이트리스트에 없는 도메인(C&C 후보)을 걸러내는 모듈.

최종 조립 — `pipeline.py`

A(`capture_during_scenario`) → B(`parse_dns`) → C(`parse_sni`) → D(`build_network_report`)를 하나로 묶는 `analyze_network(package_name, run_scenario_fn)` 진입점.

오케스트레이터(추후 `main.py`)는 이 함수 하나만 import해서 쓰면 된다.

실제 에뮬레이터 + 정상 앱으로 end-to-end 검증 완료

정상 앱(`com.google.android.calendar` , `LAUNCH_ONLY` 시나리오)으로 실제 에뮬레이터에서 캡처 → Frida 후킹 → pcap pull → DNS/SNI 파싱 → 리포트 조립까지 전체 파이프라인을 돌려서 `network_report.json` 을 검증했다 (DNS 쿼리 35건, TLS SNI 16건, 크래시 없음).

검증 중 두 가지 오탐 버그를 실데이터로 발견해서 수정함:

- `whitelist_checker.py` : `youtube.com` / `yting.com` 이 화이트리스트에 없어서 정상 앱인데도 `www.youtube.com` 이 의심 도메인으로 잡히는 문제 — 화이트리스트에 추가.
- `dns_parser.py` / `whitelist_checker.py` : Android가 캡티브 포털·DNS 하이재킹 탐지용으로 날리는 랜덤 단일 라벨 DNS 프로브가 매번 “화이트리스트 미포함”으로 잡히는 문제 — mDNS 포트(5353) 제외 + 점(.)이 없는 호스트명은 애초에 후보에서 제외하도록 수정.

▼ 설명

1. DNS 프로브 오탐 수정 부분

실제로 캡처해보니 `dgbszpfhdgn`, `tbquqcrzvpv` 같은 이상한 문자열 도메인들이 잡힘. 안드로이드 시스템 자체가 "지금 이 네트워크가 진짜 인터넷에 연결된 게 맞나, 아니면 중간에 누가 DNS를 조작해서 가짜 응답을 주는 캡티브 포털인가?"를 확인하려고, 일부러 절대 존재할 리 없는 랜덤 문자열 도메인을 만들어서 DNS 조회를 날림. 정상적인 네트워크라면 이런 도메인은 응답이 안 와야 정상이고, 만약 응답이 오면 "누가 DNS를 가로채고 있다"는 신호로 씬. 안드로이드 OS가 스스로 하는 정상적인 자가진단 트래픽.

↓

점이 없으면 애초에 진짜 인터넷 도메인이 아니니까 의심 후보에서 제외라는 필터 추가

남은 것 / 다음 논의 대상

- `suspicious.ips` 의 CDN/클라우드 IP 대역 화이트리스트

▼ 설명

1. `suspicious.ips` 가 여전히 많이 잡히는 부분

IP가 정상 도메인을 통해서 정상적으로 조회된 건지 아닌지를 전혀 안 봄. `www.google.com` 을 DNS로 정상 조회해서 나온 IP든, 앱이 도메인 조회 없이 코드에 하드코딩해놓은 수상한 IP든 상관없이, 공인 IP기만 하면 무조건 잡음. 그래서 방금 도메인 레벨에서 "애네 다 정상 구글 서버야, 의심 도메인 0건"이라고 확인한 바로 그 서버들의 IP가, IP 레벨 체크에서는 별개로 다시 "의심 후보"로 잡힘. 같은 트래픽을 도메인 기준으로 보면 정상, IP 기준으로 보면 의심 — 앞뒤가 안 맞음.

해결방안 — 논의

- (a) 이미 정상 도메인으로 확인된 IP는 IP 체크에서도 제외한다
 - (b) 구글/AWS/Cloudflare 같은 큰 클라우드/CDN 업체들의 공인 IP 대역을 별도 화이트리스트로 만들어서 대조한다
 - (c) 그냥 지금처럼 두고 "공인 IP는 일단 다 보여준다" 방식을 유지한다
- 정상 앱 검증은 Calendar 1개로 진행함 — 실제 악성 샘플(anubis.apk 등)로 돌려서 `suspicious` 판별이 의미 있게 나오는지 아직 확인 전

위험도 점수

B가 보낸 4건 질의(6주차 매핑 문서) 반영 완료.

▼ 정적분석

정적분석 점수 계산

(1) `PERMISSION_WEIGHTS` 5개뿐인 문제

Anubis류 남용 패턴(오버레이 피싱/OTP 탈취/도청·촬영/연락처 유출) 기준으로 20개로 확장.

`CAMERA` / `RECORD_AUDIO` / `READ_CONTACTS` / `ACCESS_FINE_LOCATION` 전부 포함됨.

표만 늘려서 안 끝났음 - `calculate_risk()` 가 `dangerous_permissions` (8점 이상만 걸려진 목록)만 합산해서, 8점 미만 권한은 표에 있어도 점수에 안 잡히던 버그도 같이 고침.

이제 `permissions` 전체를 봄. `dangerous_permissions` / `DANGEROUS_PERMISSION_THRESHOLD` (8)는 그대로 있으니 `permission_risk_level()` 의 high/medium 경계 로직은 안 바뀌어도 됨.

(2) `risk_score.breakdown` 근거 없던 문제

`calculate_risk_with_breakdown(manifest_data, code_data, strings_data, cert_data)` 신규 추가.

```
{
  "total": 0.54,      # calculate_risk()와 동일값
  "raw": 54,         # 정규화 전 합산 점수
  "breakdown": [
    {"factor": "android.permission.CAMERA", "weight": 7},
    {"factor": "exported_components×2 (2개)", "weight": 4},
    {"factor": "obfuscation_detected", "weight": 15},
    {"factor": "certificate.is_self_signed", "weight": 10},
    ...
  ],
}
```

`breakdown` 의 `weight` 합이 `raw` 와 항상 정확히 일치함(권한만 재현하면 합계 안 맞던 문제 해결). 권한 항목은 `factor` 에 권한 전체 이름이 그대로 들어있어서 `ABUSE_EXAMPLES` 랭 다시 매칭 가능. 어댑터에서 이 함수로 갈아타면 됨 - 기존 `calculate_risk()` 는 시그니처/ 반환형(float) 그대로라 다른 데서 쓰던 코드는 안 건드릴어도 됨.

(3) 어떤 필드를 점수에 쓸지

- `certificate.is_self_signed` → 점수에 반영함 (가중치 10). `calculate_risk()` / `calculate_risk_with_breakdown()` 둘 다 `cert_data` 인자 추가됨 (기본값 `None` 이라 안 넘겨도 동작은 함, 대신 self-signed 가산점은 안 붙음).
- `third_party_sdks` → 점수에 안 씀. 광고/분석 SDK 쓰는 것 자체는 정상 앱에도 흔해서 위험 신호로 보기 어려움. 지금처럼 어댑터에서 passthrough만 하면 됨.
- `code_analysis` / `strings.suspicious_strings` 는 기존대로 계속 씀 (변경 없음).

(4) 정적 ↔ 네트워크 교차검증

`src/cross_reference.py` (network 브랜치)에 `find_hardcoded_urls_contacted(urls, suspicious_domains)` 추가함. `strings.urls` 랑 network 모듈 `suspicious.domains` 를 호스트명 완전 일치로 대조.

아직 안 된 것

- **실제 apk 검증 전혀 안 됨** - 이 환경엔 apktool/androguard가 없어서 가짜 데이터로만 테스트함. B가 `cuckoo.apk` 로 이미 한 번 돌려봤다고 했으니, 이 브랜치 반영해서 다시 돌려봐 주면 좋을 것 같음 - 특히 `permission_risk_level()` 이 이제 CAMERA 등을 제대로 medium/high로 잡는지, `is_self_signed` 가산점이 점수에 실제로 영향 주는지 확인 필요.

▼ 검증을 통해 조정이 필요한 부분

`NORMALIZATION_CAP = 100.0` 이라는 값 자체가 여전히 근거 없는 임의의 초기값이야. 코드 주석에도 계속 "정상 앱 2~3개 vs 악성 샘플 2~3개로 실제 돌려보고 검증한 뒤 다시 맞춰야 함"이라고 적혀있어. 이번에 가중치 항목을 많이 늘렸으니 (5개 → 20개+self-signed), 위험한 앱 하나가 여러 항목에 동시에 걸리면 예전보다 훨씬 쉽게 100을 넘겨서 실제로는 "꽤 위험한 앱"이랑 "극도로 위험한 앱"이 점수상으로 똑같이 1.0으로 나와버려서, 점수가 변별력을 잃어. — 근데 이걸 "몇으로 바꿔야 맞다"는 건 실제 샘플 없이는 답이 안 나오는 문제야. 추측으로 더 건드리면 오히려 근거 없는 숫자를 근거 없는 다른 숫자로 바꾸는 것밖에 안 돼.

- `feature/static-adapter` 랑 병합 시 `manifest_parser.py` 에서 충돌 날 것 같음 - 둘 다 `PERMISSION_WEIGHTS` 표 바로 다음 줄에 각자 내용 (`DANGEROUS_PERMISSION_THRESHOLD` vs `ABUSE_EXAMPLES`) 을 추가해서 위치가 겹침. **내용 자체는 안 겹치니 병합 시 둘 다 살리면 됨.**

▼ 동적분석

동적 위험도 점수 (`risk_scorer.py`)

동적 분석으로 수집한 후킹 이벤트 (`DynamicAnalysisResult`)를 보고 "이 앱이 얼마나 위험해 보이는지"를 0.0~1.0 숫자 하나로 요약하는 모듈.

어디서 쓰이나

- `schema.py`: `DynamicAnalysisResult` 에 `risk_score: float` 필드 추가함.
- `message_parser.py`: `build_report()` 가 이벤트를 필터링한 직후 `calculate_dynamic_risk(report)` 를 호출해서 `risk_score` 를 채우고 `dynamic_report.json` 에 같이 저장함. 별도로 호출할 필요 없이 `build_report()` 만 쓰면 자동으로 들어감.

```
from dynamic_analyzer.risk_scorer import calculate_dynamic_risk

score = calculate_dynamic_risk(report) # report: DynamicAnalysisResult
```

점수 계산에 쓰는 3가지 신호

1. hook_type별 가중치 + 호출자(caller) 위치

`HookEvent.extra.caller_class` 를 보고 후킹된 함수를 누가 호출했는지 판단한다.

- `android.*`, `java.*`, `androidx.*` 등 안드로이드/자바 프레임워크 클래스면
→ 폰트 로딩, DB 쿼리 포매팅 같은 OS 내부 동작일 확률이 높음 → 가중치를

`FRAMEWORK_DISCOUNT` (0.05)만큼 대폭 할인.

- 그 외(앱 자체 패키지)면 → 앱이 직접 암호화/문자열 조립 로직을 실행했다는 뜻이라 훨씬 의미있는 신호 → 가중치에 `NON_FRAMEWORK_MULTIPLIER` (3) 곱함.

hook_type 자체의 기본 가중치는 `custom_xor` (10) > `cipher` (6) > `base64` (1) > `string_builder` (0.5) 순 — 표준 API로 못 잡는 커스텀 XOR/암호화 로직이 사람이 직접 짰 난독화·복호화 흔적일 가능성이 제일 높다고 봐서 가장 높게 잡음.

2. 평문 후보 문자열 안의 위험 키워드

`plaintext_candidates` (사람이 읽을 수 있다고 판별된 문자열들) 안에서 아래 패턴을 찾으면 건당 5점 가산:

- `http://`, `https://` (하드코딩된 URL, C2 서버 후보)
- IPv4 형태 문자열 (하드코딩된 IP)
- `password`, `token`, `secret`, `apikey` 등 (탈취 대상 키워드)
- `sms`, `contacts`, `accounts`, `clipboard` (민감 데이터 접근 흔적)

3. 노이즈 비율

`total_events_filtered / total_events_captured` 가 0.3보다 낮으면(원본 대부분이 중복이었다는 뜻) 3점 가산. 그 자체로 악성 여부를 말해주진 않지만, 반복 디코딩/조립 루프에서 흔히 관찰되는 부수적인 신호라 아주 작은 가중치만 줌.

최종적으로 세 신호를 다 더한 값을 `NORMALIZATION_CAP` (100)으로 나누고 1.0에서 clamp한다.

왜 프레임워크 호출을 할인하는가 (실측 중 발견한 문제)

처음엔 호출자 구분 없이 hook_type 가중치만 합산했는데, 정상 앱(Google Deskclock) 실측 캡처(`dynamic_report.json`)로 돌려보니 **0.63**이 나옴.

원인을 보니 `java.util.Formatter` 내부에서 발생한 `string_builder` 이벤트가 118개나 쌓여서, 개당 가중치는 0.5로 낮아도 순전히 개수만으로 점수가 부풀려진 것이었음(타임존 목록·SQL DDL 조립 같은 완전히 정상적인 동작).

즉 “몇 번 호출됐는가”보다 “누가 호출했는가”가 훨씬 중요한 신호인데, 초기 버전은 그걸 반영 못 하고 있었음. `FRAMEWORK_DISCOUNT` 를 넣어서 프레임워크발 이벤트의 영향력을 죽인 뒤 같은 데이터가 **0.03**으로 내려왔고, 앱 자체 코드에서 XOR/암호화 + C2 URL 문자열이 섞인 가짜 악성 샘플로는 **0.84**가 나오는 것까지 확인함.

검증 상태 / 미완료

- **정상 앱 1개(Deskclock 실측 캡처) + 직접 만든 가짜 악성 샘플로만 확인함.**
static 쪽과 마찬가지로 아직 1차 추측치 — 실제 공개 악성 샘플(anubis.apk 등)로 `scenario_runner.py` 를 돌려서 나온 진짜 `dynamic_report.json` 으로 가중치를 다시 맞춰야 함.
- `custom_xor` 이벤트는 아직 hooks.js에서 실제로 잡히는 게 없어서(B 작업 현황 참고 — 대상 앱 디컴파일 전이라 보류 중) 이 가중치는 아직 실데이터로 검증 못 함.
- `_FRAMEWORK_PREFIXES` 목록은 임의로 추린 것 — 실제 캡처에서 프레임워크로 분류돼야 하는데 안 되는 클래스가 나오면(서드파티 SDK 등) 목록에 추가 필요.

▼ 네트워크

네트워크 위험도 점수 계산 방법

`src/network_analyzer/network_scorer.py` 가 네트워크 분석 결과(`NetworkAnalysisResult`)를 0.0~1.0 사이의 위험도 점수로 변환하는 방법을 정리한다. `src/static_analyzer/risk_scorer.py` (정적 분석 위험도 점수)와 같은 설계를 따라서, 나중에 `main.py` 가 정적/동적/네트워크 세 점수를 같은 방식으로 다룰 수 있게 만들었다.

계산 흐름

1. 네트워크 캡처 결과(`network_report`)에서 `suspicious.domains` , `suspicious.ips` 두 항목만 본다.
2. 각 항목에 가중치(weight)를 매겨서 모두 더한 값이 raw 점수다.
3. raw 점수를 `NORMALIZATION_CAP` (100.0)으로 나누고 1.0을 넘지 않게 자른다.

```
raw = sum(각 suspicious 항목의 weight)
total = min(raw / 100.0, 1.0)
```

어떤 항목이 suspicious로 잡히는가

`suspicious.domains` 와 `suspicious.ips` 는 `network_scorer.py` 가 직접 만드는 게 아니라 그 앞 단계인 `whitelist_checker.py` , `ip_checker.py` 가 채워서 넘겨준다.

- `suspicious.domains` (`whitelist_checker.find_suspicious_domains`): DNS 쿼리/TLS SNI로 관찰된 도메인 중 화이트리스트(`WHITELIST_DOMAINS` , Google/광고 SDK/분석 툴/CDN 등 ~40개)에 없는 도메인. 이 중 도메인 필드에 호스트명 대신 IP가 그대로 들어있는 경우는 "하드코딩된 형태"로 따로 표시된다.
- `suspicious.ips` (`ip_checker.find_suspicious_ips`): DNS로 얻은 응답 IP나 TLS 목적지 IP 중 사설/루프백/링크로컬/멀티캐스트가 아닌 공인 IP 전부.

가중치

항목	가중치	근거
하드코딩된 IP 도메인 (<code>HARDCODED_IP_DOMAIN_WEIGHT</code>)	20	도메인 필드에 DNS 조회 없이 IP가 그대로 박혀 있음 → 도메인 기반 차단 (블랙/화이트리스트)을 우회하려는 전형적인 C&C 패턴이라 가장 강한 신호로 취급
미화이트리스트 도메인 (<code>UNLISTED_DOMAIN_WEIGHT</code>)	8	화이트리스트가 ~40개짜리 1차 목록이라 놓친 정상 도메인일 가능성도 있어 IP 리터럴보다 약하게 잡음
의심 IP 1건 (<code>SUSPICIOUS_IP_WEIGHT</code>)	1	<code>ip_checker</code> 의 "공인 IP면 전부 의심" 규칙 자체가 노이즈가 많은 1차 버전이라(자체 docstring에 명시), 해당 가중치를 낮게 잡아야 정상 앱 점수가 안 부풀려짐

세 값 모두 실제 정상/악성 샘플 트래픽으로 검증되지 않은 1차 휴리스틱이다. 특히 `SUSPICIOUS_IP_WEIGHT` 를 의도적으로 낮게 잡은 이유는, 정상 앱(Calendar)을 실행해 검증했을 때도 공인 IP라는 이유만으로 17건이 `suspicious.ips` 에 잡혔기 때문이다 — 해당 가중치를 1로 낮춰야 이런 정상 트래픽이 점수를 과도하게 부풀리지 않는다.

검증 예시 (`python -m src.network_analyzer.network_scorer`)

정상 앱 (Calendar, 실패처) — `suspicious.domains` 0건, `suspicious.ips` 17건(공인 IP 노이즈):

```
raw = 17 * 1 = 17
total = 17 / 100 = 0.17
```

의심 앱 (C2 후보 포함, 가상 시나리오) — 미화이트리스트 도메인 1개 + 하드코딩된 IP 도메인 1개 + 의심 IP 1건:

```
raw = 8(미화이트리스트) + 20(하드코딩 IP) + 1(의심 IP) = 29
total = 29 / 100 = 0.29
```

breakdown (근거 포함 버전)

점수만으로는 왜 그 점수가 나왔는지 알 수 없어서, `calculate_network_risk_with_breakdown()` 은 각 항목이 몇 점을 기여했는지 리스트로 함께 반환한다.

```
{
  "total": 0.29,          # calculate_network_risk()와 동일한 값
  "raw": 29,             # 정규화 전 합산 점수
  "breakdown": [
    {"factor": "미화이트리스트 도메인: evil-c2.example.net", "weight": 8},
    {"factor": "하드코딩된 IP 도메인: 1.2.3.4", "weight": 20},
    {"factor": "의심 IP: 1.2.3.4", "weight": 1},
  ],
}
```

`calculate_network_risk()` 와 `calculate_network_risk_with_breakdown()` 은 내부적으로 같은 `_score_breakdown()` 함수를 공유한다 — 실제 점수 계산에 쓰인 항목과 `breakdown`에 보여주는 근거가 항상 일치하게 하기 위해서다.

한계 / 앞으로 검증 필요한 것

- 화이트리스트(~40개 도메인)와 IP 판별 규칙("공인 IP면 전부 의심") 둘 다 1차 휴리스틱이라 실제 정상 앱/악성 샘플 트래픽으로 가중치가 적절한지 검증된 적이 없다.
- `anubis.apk` / `syssecapp.apk` 같은 실제 악성 샘플과 정상 앱 여러 개를 같은 방식으로 캡처해서 점수 분포를 비교하는 작업이 남아있다.
- 정적 분석 위험도 점수와 마찬가지로, 이 점수가 최종적으로 정적/동적 점수와 어떻게 합산될지는 `main.py` 통합 파이프라인(7~8주차 범위)에서 결정된다.